

Predation rates based on dummy caterpillars

Maryam Gheibi, Mozhdeh Ramezani Dastjerdi , Shirin Shoghli, Shahrzad Torabi, Sanaz Bayat,
February 08, 2024

Task 6 (January 11 - February 8)

First Dataset: <https://www.bexis.uni-jena.de/ddm/data/Showdata/31113>

Categorical Attributes:

Nominal data:

- Dummy_ID
- Region
- PlotID
- Bexis_PlotID
- Treatment
- Subplot
- Predation_total
- Dummy

Ordinal data:

- Subplot

Binary data:

- Arthropod_mark
- Bird_mark
- Mammal_mark
- Gastropod_mark
- Dummy_missing

Non-categorical Data:

Discrete Data

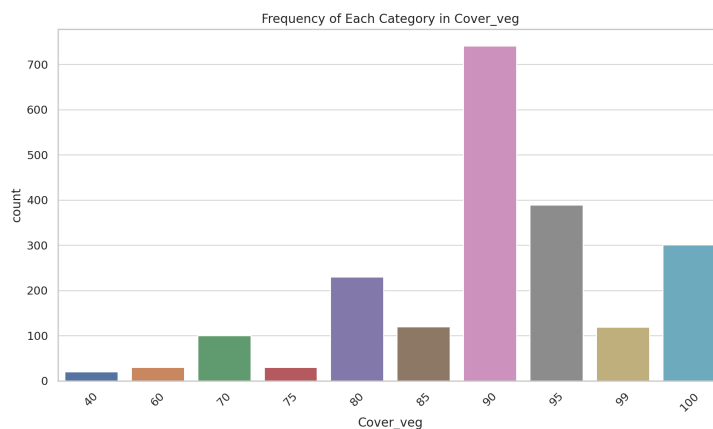
- Cover_veg
- Cover_canopy

Continuous Data

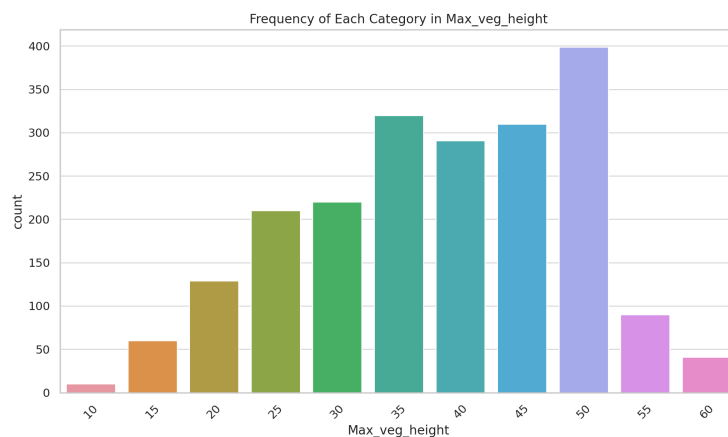
- Max_veg_height
- Average_veg_height
- Start_date
- End_date

Visualization

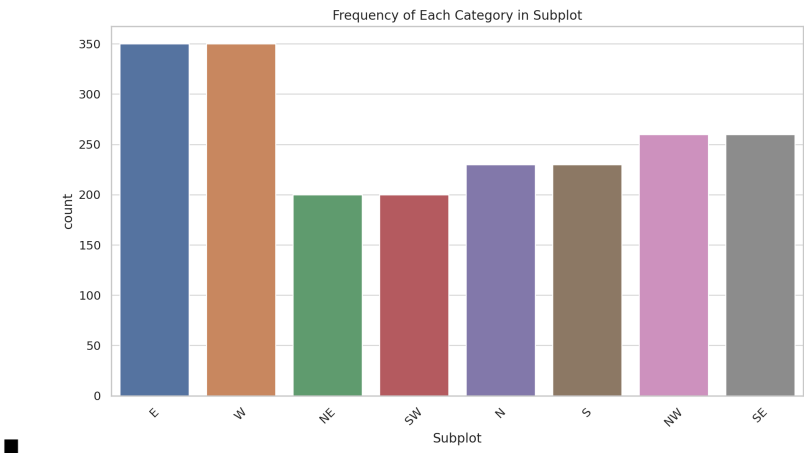
The Cover_veg plot illustrates the distribution of vegetation cover percentages, essential for understanding the extent of vegetation coverage and identifying areas with varying levels of vegetation.



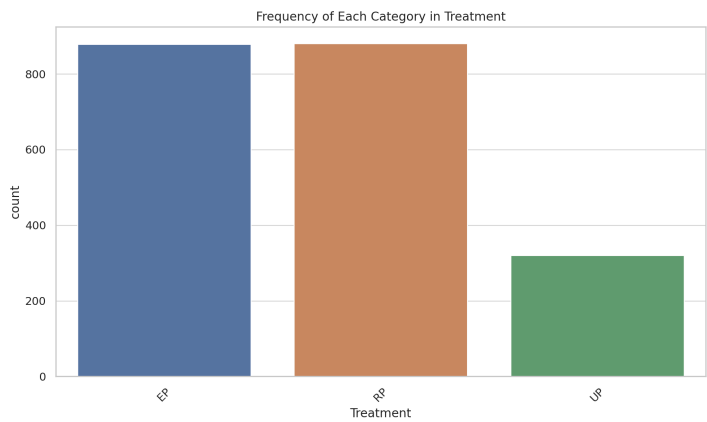
The Max_veg_height histogram depicts the distribution of maximum vegetation height, highlighting the variability and common vegetation heights in the study areas.



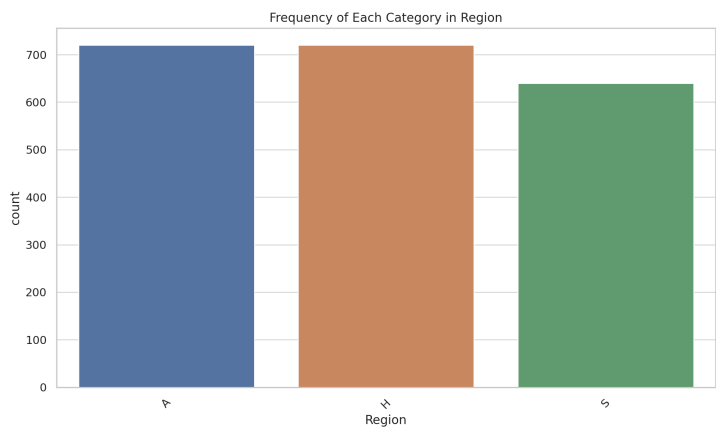
The Subplot plot shows the frequency of observations in subplots, offering insights into the experimental layout and spatial distribution of data.



The Treatment plot illustrates the distribution of different treatments applied, aiding in understanding the experimental design and treatment distribution.



The Region plot reveals the frequency of observations across regions, identifying areas with higher concentrations of data points.



Implementation

```
import matplotlib.pyplot as plt

import seaborn as sns

# Setting the aesthetic style of the plots

sns.set(style="whitegrid")

# Define a function to create plots for both categorical and numeric attributes

def create_plots(data, attributes):

    for attribute in attributes:

        if data[attribute].dtype == 'object' or len(data[attribute].unique()) <= 20:

            # Categorical data or numeric with limited unique values

            plt.figure(figsize=(10, 6))

            sns.countplot(x=attribute, data=data)

            plt.title(f'Frequency of Each Category in {attribute}')

            plt.xticks(rotation=45)

            plt.tight_layout()

        else:

            # Numeric data

            plt.figure(figsize=(10, 6))

            sns.histplot(data[attribute], kde=True, bins=30)

            plt.title(f'Distribution of {attribute}')
```

```
plt.tight_layout()

plt.show()

# List of attributes to plot

attributes_to_plot = ['Region', 'Treatment', 'Subplot', 'Max_veg_height', 'Cover_veg']

# Generate plots

create_plots(data, attributes_to_plot)
```

Second Dataset: <https://www.bexis.uni-jena.de/ddm/data/Showdata/24246>

Categorical Attributes:

Nominal data:

- Exploratorium
- PlotID
- PlotID2
- Position
- PredationType

Binary data:

- Side

Non-categorical Data:

Continuous Data

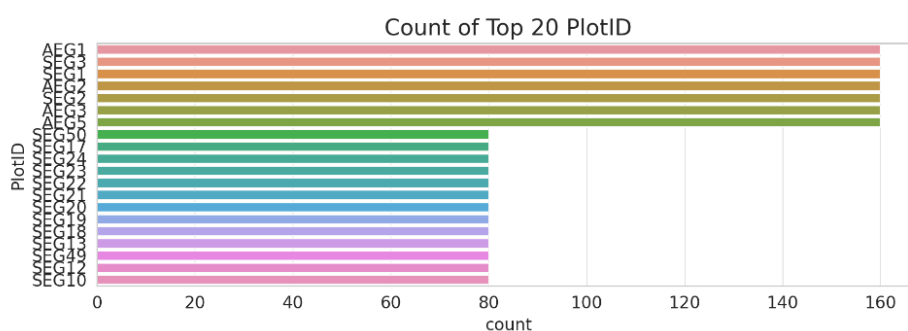
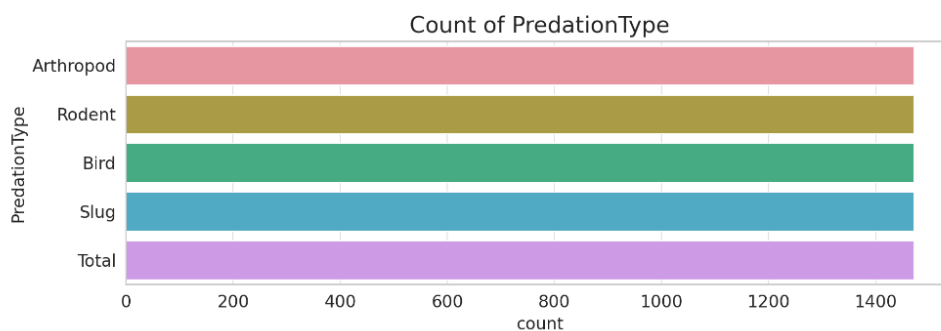
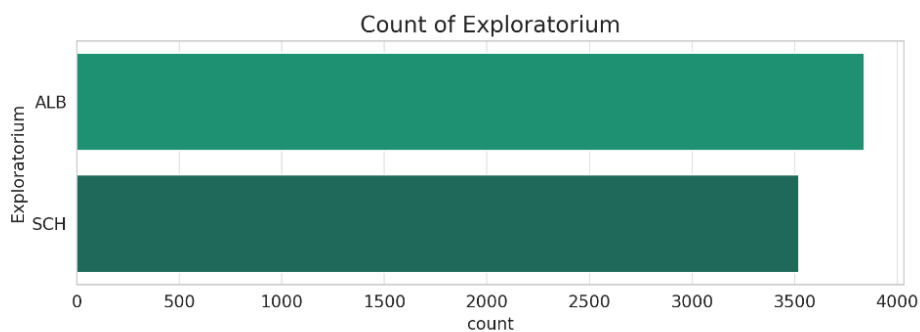
- PredationMark

Visualization

What are count plots?

The count plots for categorical columns show the distribution of each category within those columns. For columns with many unique values, such as "PlotID" and "PlotID2".

These plots help in understanding the frequency of each category and can indicate whether a variable might be nominal or binary. For example, if a column had only two bars of significant height, it might be considered binary.



implementation

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the aesthetic style of the plots
sns.set_style("whitegrid")

# Determine the number of unique values in each categorical column to decide on the plot types
unique_values = dataset.select_dtypes(include=['object']).nunique()

# Plot for categorical variables with relatively few unique values
fig, axs = plt.subplots(nrows=len(unique_values), figsize=(10, 20))

for i, column in enumerate(unique_values.index):
    # For columns with many unique values, we use a count plot of the top N categories
    if unique_values[column] > 20:
        top_n = dataset[column].value_counts().head(20)
        sns.countplot(y=column, data=dataset[dataset[column].isin(top_n.index)], ax=axs[i], order=top_n.index)
        axs[i].set_title(f'Count of Top 20 {column}')
    else:
        sns.countplot(y=column, data=dataset, ax=axs[i])
        axs[i].set_title(f'Count of {column}')

plt.tight_layout()

# Plot for the numeric column 'PredationMark' to show its distribution
plt.figure(figsize=(10, 6))
sns.histplot(dataset['PredationMark'].dropna(), kde=True, bins=30)
plt.title('Distribution of PredationMark')
plt.xlabel('PredationMark')
plt.ylabel('Frequency')

plt.show()
```

Third Dataset: <https://www.bexis.uni-jena.de/ddm/data/Showdata/25807>

Categorical Attributes:

Nominal data:

- PlotID
- Region
- Subplot

Ordinal data:

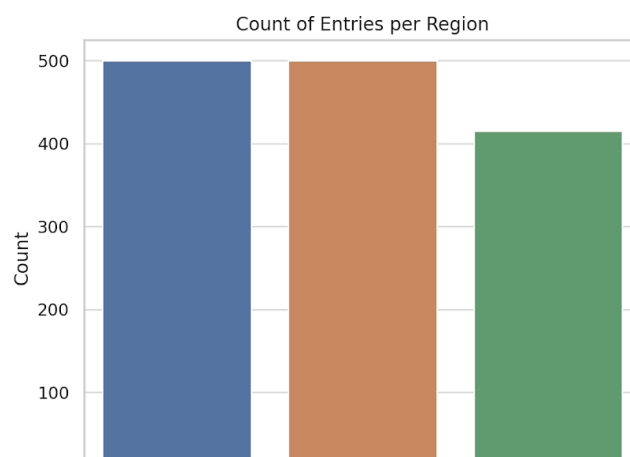
- Subplot

Non-categorical Data:

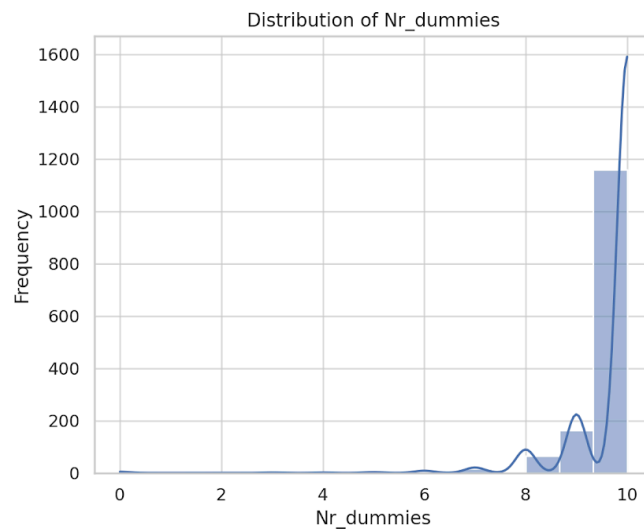
Continuous Data

- Date_start
- Time_start
- Date_end
- Time_end
- Nr_dummies
- Arthropod_prop
- Bird_prop

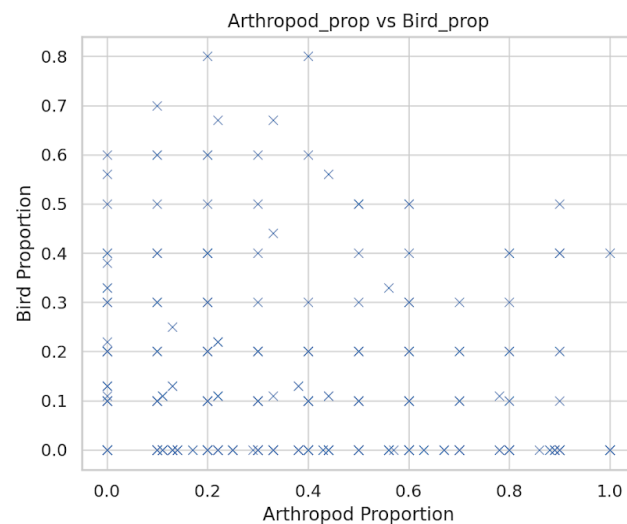
In This bar chart we show the frequency of entries for each region in the dataset, offering insights into the distribution of data across different regions.



In This histogram illustrates we can see the distribution of the number of dummies (`Nr_dummies`) used in the experiments. The presence of a kernel density estimate (KDE) curve also gives an idea of the distribution's shape.



And the scatter plot explores the relationship between the proportion of dummies with marks by arthropods (`Arthropod_prop`) and those with marks by birds (`Bird_prop`). Each point represents an observation in the dataset, allowing for the identification of any patterns or correlations between these two variables.



implementation

```
import pandas as pd

# Define the path to the CSV dataset file
csv_file_path = os.path.join(extracted_folder_path, '25807_3_data.csv')

# Load the dataset into a DataFrame
data_df = pd.read_csv(csv_file_path)

# Display basic information about the dataset and the first few rows to understand its structure
data_df.info(), data_df.head()

import matplotlib.pyplot as plt
import seaborn as sns

# Set the style of seaborn
sns.set(style="whitegrid")

# Plotting
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Bar chart for the 'Region' attribute
sns.countplot(x='Region', data=data_df, ax=axes[0])
axes[0].set_title('Count of Entries per Region')
axes[0].set_xlabel('Region')
axes[0].set_ylabel('Count')

# Histogram for the 'Nr_dummies' attribute
sns.histplot(data_df['Nr_dummies'], kde=True, bins=15, ax=axes[1])
axes[1].set_title('Distribution of Nr_dummies')
axes[1].set_xlabel('Nr_dummies')
axes[1].set_ylabel('Frequency')

# Scatter plot for 'Arthropod_prop' vs 'Bird_prop'
sns.scatterplot(x='Arthropod_prop', y='Bird_prop', data=data_df, ax=axes[2])
axes[2].set_title('Arthropod_prop vs Bird_prop')
axes[2].set_xlabel('Arthropod Proportion')
axes[2].set_ylabel('Bird Proportion')

plt.tight_layout()
plt.show()
```

Contributions

Mozhdeh Ramezani Dastjerdi

In this task we download related datasets from the source at the first step, In the second one we start to analyze datasets based on category types which are divided into two main categories Categorical Attributes and Non Categorical Attributes.

As we mentioned in previous parts each category has different subcategories that we divide based on their type. Such as Nominal data, Ordinal data, and Binary data.

In the last step, we start to draw plots depending on the attribute type. So we implement the codes in Python with the help of Pandas and Matplotlib, etc.

We also put all of codes that we implement in details in previous parts.

Maryam Gheibi

In this task we download related datasets from the source at the first step, In the second one we start to analyze datasets based on category types which are divided into two main categories Categorical Attributes and Non Categorical Attributes.

As we mentioned in previous parts each category has different subcategories that we divide based on their type. Such as Nominal data, Ordinal data, and Binary data.

In the last step, we start to draw plots depending on the attribute type. So we implement the codes in Python with the help of Pandas and Matplotlib, etc.

We also put all of codes that we implement in details in previous parts.

Shirin Shoghli

In this task we download related datasets from the source at the first step, In the second one we start to analyze datasets based on category types which are divided into two main categories Categorical Attributes and Non Categorical Attributes.

As we mentioned in previous parts each category has different subcategories that we divide based on their type. Such as Nominal data, Ordinal data, and Binary data.

In the last step, we start to draw plots depending on the attribute type. So we implement the codes in Python with the help of Pandas and Matplotlib, etc.

We also put all of codes that we implement in details in previous parts.

Shahrzad Torabi

In this task we download related datasets from the source at the first step, In the second one we start to analyze datasets based on category types which are divided into two main categories Categorical Attributes and Non Categorical Attributes.

As we mentioned in previous parts each category has different subcategories that we divide based on their type. Such as Nominal data, Ordinal data, and Binary data.

In the last step, we start to draw plots depending on the attribute type. So we implement the codes in Python with the help of Pandas and Matplotlib, etc.

We also put all of codes that we implement in details in previous parts.

Sanaz Bayat

In this task we download related datasets from the source at the first step, In the second one we start to analyze datasets based on category types which are divided into two main categories Categorical Attributes and Non Categorical Attributes.

As we mentioned in previous parts each category has different subcategories that we divide based on their type. Such as Nominal data, Ordinal data, and Binary data.

In the last step, we start to draw plots depending on the attribute type. So we implement the codes in Python with the help of Pandas and Matplotlib, etc.

We also put all of codes that we implement in details in previous parts.